Lab 01: Reverse engineer x86 assembly code

Ahmed Mahfooz Ali Khan BTHBI338

Task 1:

Write C-pseudocode for assembly code shown below. You should be able to recognize specific C-constructs such as assignmens, loops, conditionals and functions.

```
01: push ebp
02: mov ebp, esp
03: mov eax, [ebp+8]
04: sub eax, 41h
05: jz short loc_caseA
06: dec eax
07: jz short loc_caseB
08: dec eax
09: jz short loc_caseC
10: mov al, 5Ah
11: movzx eax, al
12: pop ebp
13: retn
14: loc_caseC:
15: mov al, 43h
16: movzx eax, al
17: pop ebp
18: retn
19: loc_caseB:
20: mov al, 42h
21: movzx eax, al
22: pop. ebp
23: retn
24: loc_caseA:
25: mov al, 41h
26: movzx eax, al
```

27: pop ebp
28: retn

Explanation of the Assembly Code

1. **Function Prologue and Initialization**:
    - push ebp and mov ebp, esp: Standard function prologue to set up the stack frame. ○ mov eax, [ebp+8]: Load the input parameter into the eax register (function argument input).
2. **Conditionals**:
    - sub eax, 41h: Subtract 0x41 from the input (temp = input - 0x41).
    - jz short loc_caseA: Jump to loc_caseA if the result is zero (temp == 0).
    - dec eax: Decrement eax (temp--).
    - jz short loc_caseB: Jump to loc_caseB if the result is zero (temp == 0).
    - dec eax: Decrement eax again (temp--).
    - jz short loc_caseC: Jump to loc_caseC if the result is zero (temp == 0).
3. **Return Values**:
    - mov al, 41h, mov al, 42h, mov al, 43h, mov al, 5Ah: Set the return values ('A', 'B', 'C', 'Z').
    - movzx eax, al: Zero-extend the result to 32 bits.
    - pop ebp, retn: Function epilogue to restore the stack and return to the caller. **Output Behavior**

- If the input equals 0x41 (65), the function returns 'A'.
- If the input equals 0x42 (66), the function returns 'B'.
- If the input equals 0x43 (67), the function returns 'C'.
- For all other inputs, the function returns 'Z'.

C-Pseudocode for the Given Assembly Code

Below is the C-pseudocode that maps the provided assembly instructions. It recognizes key constructs like variable assignments, conditionals, and functions:

```
char process_input(int input) {
   // Subtract 0x41 from the
input     int temp = input -
0x41;     // Check if temp ==
0 (case A)     if (temp == 0) {
return 'A'; // ASCII 0x41
   }
   // Decrement temp and check if temp == 0
(case B)     temp--;     if (temp == 0) {         return
'B'; // ASCII 0x42
   }
   // Decrement temp and check if temp == 0
(case C)     temp--;     if (temp == 0) {         return
'C'; // ASCII 0x43
   }
   // Default case
return 'Z'; // ASCII
0x5A
}
```

Task 2:

Write C-pseudocode for assembly code shown below. You should be able to recognize specific C-constructs such as assignmens, loops, conditionals and functions.

```
01: cmp edi, 5
02: ja short loc_10001141
03: jmp ds:off_100011A4[edi*4]
```

04: loc_10001125:

05: mov esi, 40h

06: jmp short loc_10001145

07: loc_1000112C:

08: mov esi, 25h

09: jmp short loc_10001145

10: loc_10001133:

11: mov esi, 39h

12: jmp short loc_10001145

13: loc_1000113A:

14: mov esi, 10h

15: jmp short loc_10001145

16: loc_10001141:

17: mov esi,

[esp+0Ch]

18: ...

19: off_100011A4:

20. dd offset loc_10001125

21: dd offset loc_10001125

22: dd offset loc_1000113A

23: dd offset loc_1000112C

24: dd offset loc_10001133

25: dd offset loc_1000113A

Explanation of the Assembly Code

The assembly function behaves as a decision-making system based on the input edi:

1. If edi > 5, it uses the fallback value stack_input.
2. If edi <= 5, it uses a jump table to handle cases (0 to 5) with specific values for esi:

- 0, 1 → esi = 0x40;
- 2, 5 → esi = 0x10;
- 3 → esi = 0x25;
- 4 → esi = 0x39;

This structure is implemented efficiently using a combination of conditionals and indirect jumps.

C-Pseudocode for the Given Assembly Code

Here's the C pseudocode for the assembly snippet, recognizing key C constructs like conditionals, arrays, and jumps:

```
int process_input(int edi, int stack_input) {
   int esi;
   // If edi > 5, use
stack_input     if (edi > 5)
{        esi = stack_input;
   } else {
      // Array of function pointers (indirect jump table)
      void* jump_table[] = {loc_10001125, loc_10001125, loc_1000113A, loc_1000112C,
loc_10001133, loc_1000113A};
      // Jump to the appropriate location based on
edi        switch (edi) {            case 0:
case 1:
            // loc_10001125              esi = 0x40; // Value
corresponding to loc_10001125
            break;
case 2:
```

```
            // loc_1000113A                esi = 0x10; // Value
corresponding to loc_1000113A
            break;
          case 3:
            // loc_1000112C                esi = 0x25; // Value
corresponding to loc_1000112C
            break;
case 4:
            // loc_10001133                esi = 0x39; // Value
corresponding to loc_10001133
            break;
case 5:
            // loc_1000113A                esi = 0x10; // Value
corresponding to loc_1000113A
            break;          default:
esi = 0; // Default safety case
break;
      }
}
return
esi;
}
```

Task 3:

The code below calls a function. What type of calling convention is being used? What are the fundamental operations of the call convention you have identified?

```
01: 00401002 mov edi, ds:printf
02: 00401008 xor esi, esi { Clear esi (set it to 0)}
```

03: 0040100A lea ebx, [ebx+0]

04: 00401010 loc_401010:

05: 00401010 push esi { Push the value of esi}

06: 00401011 push offset StrFormat ; "%d\n" { Push the format string "%d\n"}

07: 00401016 call edi ; printf { Call the printf function}

08: 00401018 inc esi { Increment loop counter}

09: 00401019 add esp, 8 { Clean up the stack (2 arguments × 4 bytes each)}

10: 0040101C cmp esi, 0Ah. { Compare esi with 10}

11: 0040101F jl short loc_401010 { Jump back to the loop if esi < 10}

12: 00401021 push offset aEnd ; "end\n" { Push the string "end\n"}

13: 00401026 call edi ; printf { Call the printf function}

14: 00401028 add esp, 4 { Clean up the stack (1

argument)} **Solution:**

- **Calling Convention:** cdecl
- **Fundamental Operations:**
    1. **Arguments are pushed onto the stack in reverse order.**
    2. **The caller (main) cleans up the stack after the function call.**
    3. **Control is transferred using call, and stack adjustment is performed after the function returns.**
    4. The convention allows for variable argument functions, making it suitable for functions like printf.

C-Pseudocode for the Given Assembly Code

#include <stdio.h> void main() { int esi = 0; // Loop

counter, equivalent to `esi` in the assembly

// Loop to print numbers from 0 to 9 while (esi

< 10) { printf("%d\n", esi); // Print the current

value of `esi` esi++; // Increment the

loop counter

```
    }
    // Print "end" after the loop finishes
    printf("end\n");
}
```

Task 4:

The assembly code shown below belongs to a C-function called from main(). The function takes two parameters: a pointer to an array and the length of the array. Assume that when the function is called the first argument points to a byte array containing the following values in sequence: 21, 7, 0, 16, 10, 12, 18. The second argument is set to the length of the array, that is equal to value 7 in this case.

Write C-pseudocode for assembly code shown below. You should be able to recognize specific C-constructs such as assignmens, loops, conditionals and functions.

The puts() function called att offset 0x6D5 prints out a word from the English language. What word is it? The only argument to puts() is pushed on the stack on the line before.

```
.text:00000627 ; ||||||||||||||| S U B R O U T I N E |||||||||||||||||||||||||||||||||||||
.text:00000627
.text:00000627 ; Attributes: bp-based frame
.text:00000627
.text:00000627 public _Z1fPhj
.text:00000627 _Z1fPhj proc near ; CODE XREF: main+53 p
.text:00000627
.text:00000627 var_5C = dword ptr -5Ch
.text:00000627 var_50 = dword ptr -50h
.text:00000627 var_4C = dword ptr -4Ch
.text:00000627 var_47 = dword ptr -47h
.text:00000627 var_43 = dword ptr -43h
.text:00000627 var_3F = dword ptr -3Fh
```

```
.text:00000627 var_3B = dword ptr -3Bh
.text:00000627 var_37 = dword ptr -37h
.text:00000627 var_33 = dword ptr -33h
.text:00000627 var_2F = word ptr -2Fh
.text:00000627 var_2D = byte ptr -2Dh
.text:00000627 var_2C = byte ptr -2Ch
.text:00000627 var_C = dword ptr -0Ch
.text:00000627 var_4 = dword ptr -4
.text:00000627 arg_0 = dword ptr 8
.text:00000627 arg_4 = dword ptr 0Ch
.text:00000627
.text:00000627 push ebp
.text:00000628 mov ebp, esp
.text:0000062A push ebx
.text:0000062B sub esp, 64h
.text:0000062E call __x86_get_pc_thunk_bx
.text:00000633 add ebx, 199Dh
.text:00000639 mov eax, [ebp+arg_0]
.text:0000063C mov [ebp+var_5C], eax
.text:0000063F mov eax, large gs:14h
.text:00000645 mov [ebp+var_C], eax
.text:00000648 xor eax, eax
.text:0000064A mov [ebp+var_47], 'DCBA'
.text:00000651 mov [ebp+var_43], 'HGFE'
.text:00000658 mov [ebp+var_3F], 'LKJI'
.text:0000065F mov [ebp+var_3B], 'PONM'
.text:00000666 mov [ebp+var_37], 'TSRQ'
```

```
.text:0000066D mov [ebp+var_33], 'XWVU'
.text:00000674 mov [ebp+var_2F], 'ZY' .text:0000067A mov [ebp+var_2D], 0
.text:0000067E sub esp, 4
.text:00000681 push 20h ; size_t
.text:00000683 push 0 ; int
.text:00000685 lea eax, [ebp+var_2C]
.text:00000688 push eax ; void *
.text:00000689 call _memset
.text:0000068E add esp, 10h
.text:00000691 mov [ebp+var_50], 0
.text:00000698
.text:00000698 loc_698: ; CODE XREF: _Z1fPhj+A5j
.text:00000698 mov eax, [ebp+var_50]
.text:0000069B cmp [ebp+arg_4], eax
.text:0000069E jbe short loc_6CE
.text:000006A0 mov edx, [ebp+var_50]
.text:000006A3 mov eax, [ebp+var_5C]
.text:000006A6 add eax, edx
.text:000006A8 movzx eax, byte ptr [eax]
.text:000006AB movzx eax, al
.text:000006AE mov [ebp+var_4C], eax
.text:000006B1 mov edx, [ebp+var_4C]
.text:000006B4 mov eax, [ebp+var_50]
.text:000006B7 add eax, edx
.text:000006B9 movzx eax, byte ptr [ebp+eax+var_47]
.text:000006BE lea ecx, [ebp+var_2C]
.text:000006C1 mov edx, [ebp+var_50]
```

```
.text:000006C4 add edx, ecx
.text:000006C6 mov [edx], al
.text:000006C8 add [ebp+var_50], 1
.text:000006CC jmp short loc_698
.text:000006CE ; ---------------------------------------------------------------------------
.text:000006CE
.text:000006CE loc_6CE: ; CODE XREF: _Z1fPhj+77 j
.text:000006CE sub esp, 0Ch
.text:000006D1 lea eax, [ebp+var_2C]
.text:000006D4 push eax ; char *
.text:000006D5 call _puts
.text:000006DA add esp, 10h
.text:000006DD mov eax, 0
.text:000006E2 mov ecx, [ebp+var_C]
.text:000006E5 xor ecx, large gs:14h
.text:000006EC jz short loc_6F3
.text:000006EE call __stack_chk_fail_local
.text:000006F3
.text:000006F3 loc_6F3: ; CODE XREF: _Z1fPhj+C5 j
.text:000006F3 mov ebx, [ebp+var_4]
.text:000006F6 leave
.text:000006F7 retn
.text:000006F7 _Z1fPhj endp
```

C-Pseudocode for the Assembly Function

```
#include <stdio.h>
#include <string.h>
```

```
void f(unsigned char *array, int length) {
   // Lookup table initialization
   char lookup_table[28] = {
      'A', 'B', 'C', 'D',
      'E', 'F', 'G', 'H',
      'I', 'J', 'K', 'L',
      'M', 'N', 'O', 'P',
      'Q', 'R', 'S', 'T',
      'U', 'V', 'W', 'X',
      'Y', 'Z', '\0'
   };
   // Result array initialization
   char result[32];
   memset(result, 0, sizeof(result)); // Initialize result array with zeros
   // For loop to process the input array
   for (int i = 0; i < length; i++) {
      unsigned char index = array[i];    // Get the value at index 'i' from the input array
      unsigned char value = lookup_table[index]; // Use the value as an index to retrieve the corresponding character from the lookup table
      result[i] = value;                // Store the retrieved character in the 'result' array at the same index               // Store mapped value in result
   }
// Print the mapped result string using puts
   puts(result);
}
```

Explanation of Pseudocode:

1. Lookup Table Initialization (lookup_table):

- The array lookup_table represents the series of characters being loaded in the assembly:
    - 'A' to 'Z' and a null terminator '\0'.
- This corresponds to the series of mov instructions:
- mov [ebp+var_47], 'DCBA'
- mov [ebp+var_43], 'HGFE'
- mov [ebp+var_3F], 'LKJI'
- mov [ebp+var_3B], 'PONM'
- mov [ebp+var_37], 'TSRQ'
- mov [ebp+var_33], 'XWVU'
- mov [ebp+var_2F], 'ZY'

mov [ebp+var_2D], 0

2. Result Array Initialization (result):

- A result array is created and initialized with zeros using memset.
- This corresponds to:
- lea eax, [ebp+var_2C]
- push eax
- call _memset

3. For Loop (for):

- The loop iterates over the length of the input array:
    - It checks the condition i < length using:
    - cmp [ebp+arg_4], eax
    - jbe short loc_6CE

4. Character Mapping (lookup_table[index]):

- The input byte array[i] is used as an index to map values using the lookup table:
- mov edx, [ebp+var_50] ; Load loop index
- mov eax, [ebp+var_5C] ; Load base address of array

- add eax, edx ; Calculate array[i]
- movzx eax, byte ptr [eax] ; Load value from array[i]
- mov [ebp+var_4C], eax ; Store value
- add eax, edx ; Add value

movzx eax, byte ptr [ebp+eax+var_47] ; Map using lookup table

5. Store Mapped Value in result:

- The mapped value is stored in the result array:
- lea ecx, [ebp+var_2C] ; Base address of result array

mov [edx + ecx], al ; Store mapped value at result[i]

6. Print Result (puts):

- The puts function is called to print the result:
- lea eax, [ebp+var_2C]
- push eax

call _puts

Given Input and Expected Output:

Input Array:

{21, 8, 2, 19, 14, 17, 24}

Lookup Table Mapping:

- Index 21 maps to '**V**'
- Index 8 maps to **'I'**
- Index 2 maps to '**C**'
- Index 19 maps to '**T**'
- Index 14 maps to '**O**'
- Index 17 maps to '**R**'
- Index 24 maps to **'Y'**

Final Output Word:

"VICTORY"

Conclusion

This assignment reinforced the understanding of:

- Translating assembly instructions into high-level descriptions.
- Designing and analyzing algorithms using pseudocode.
- Comparing assembly language and high-level implementations in terms of performance and complexity.

The provided tasks demonstrated the versatility of assembly in low-level programming and the importance of abstraction for algorithm design.

References

1. Class lectures and videos
2. Chatgpt: https://chatgpt.com/share/67433bc9-99c4-800d-8c3c-c49a33a16229